

Foundations of Data Science for Biologists

Working with text in Unix

BIO 724D

2026-JAN-27

Instructors: Greg Wray and Kayla Wilhoit

Learning goals for today

More background on Unix

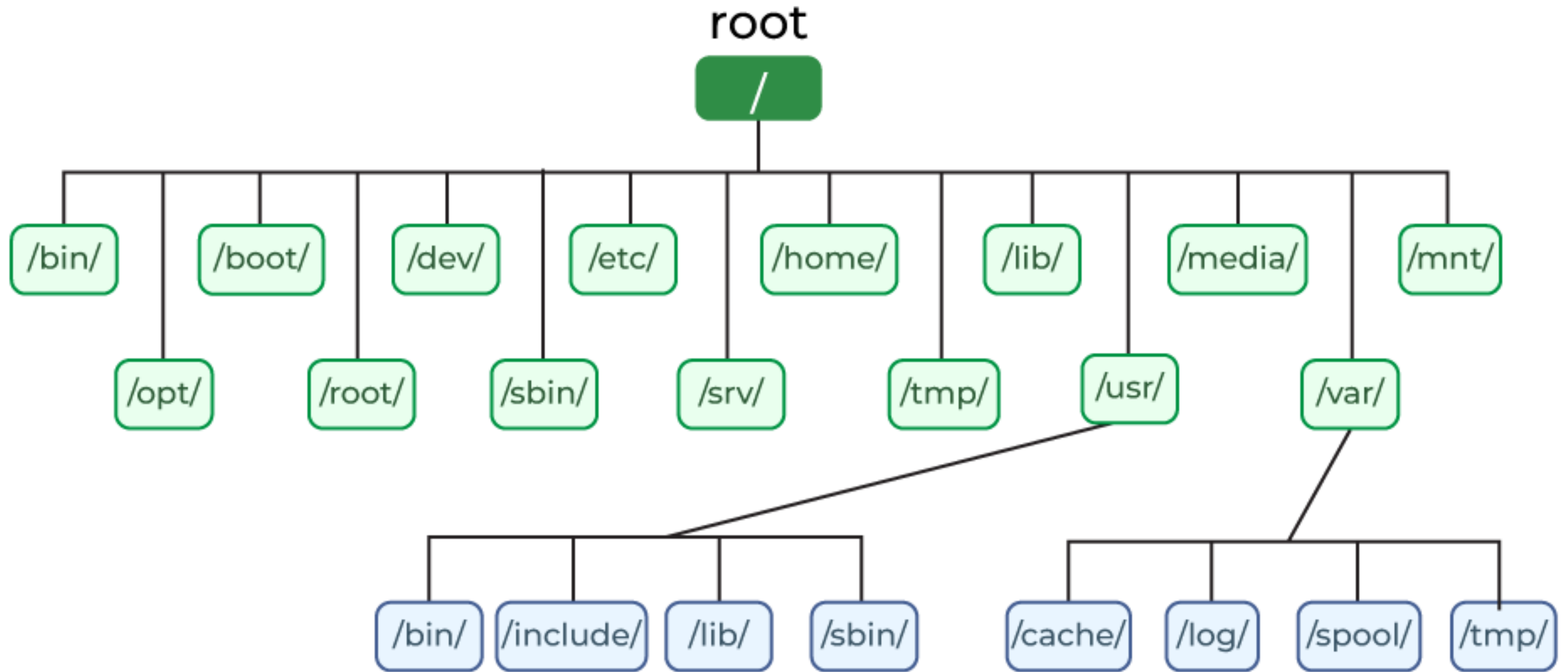
Text encodings

Using Unicode, expansions, and regular expressions in Unix

sed stream editor

The Unix file system

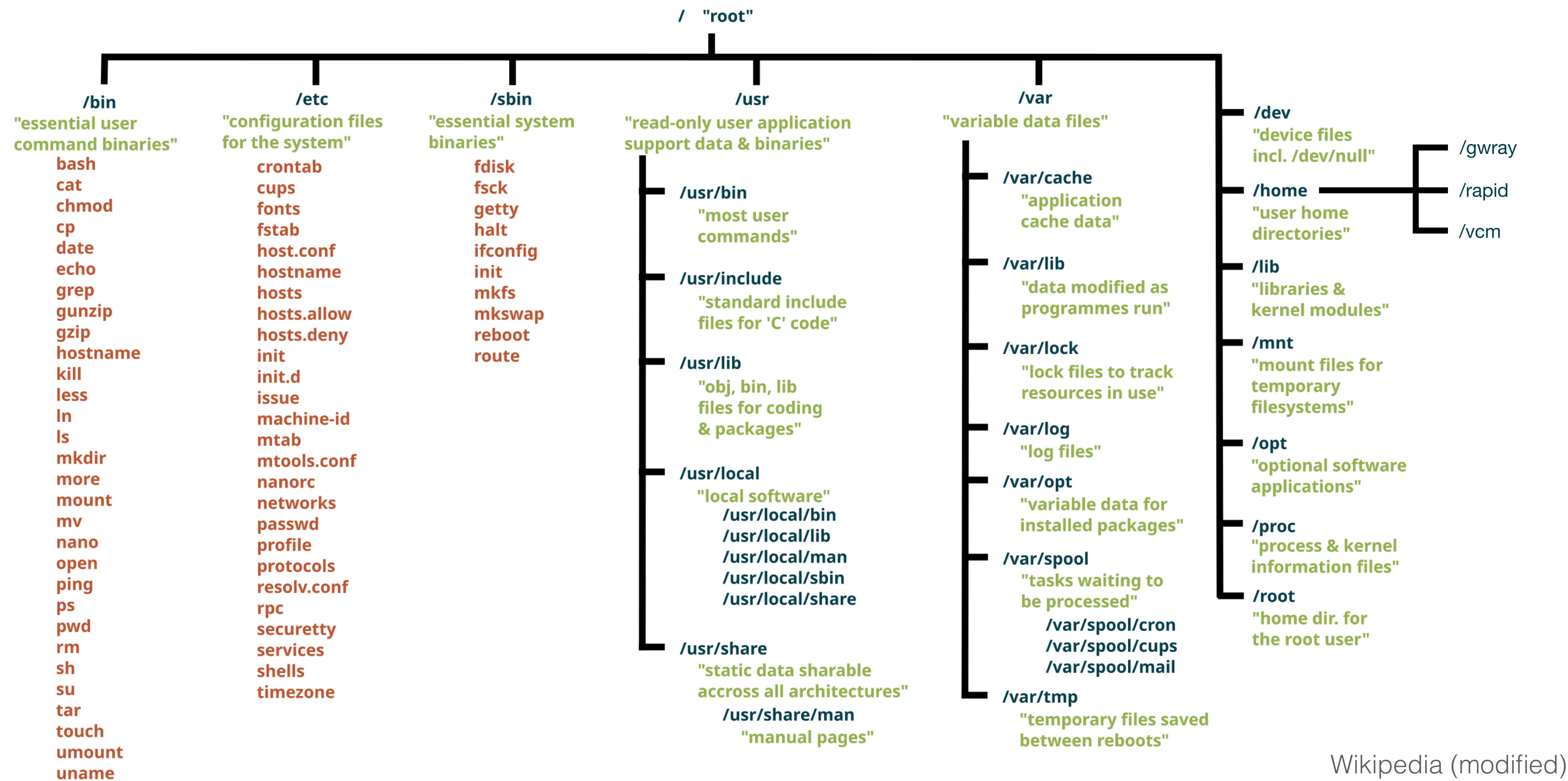
Standard Unix file structure



Key directories in the Unix file system

/	root directory of the entire file system
/bin	essential executable commands / binaries for all users
/dev	device drivers: special files representing printers, extra disks, etc.
/etc	system-specific and configuration files
/home	home directories of users
/lib	essential files needed by programs in /bin and /sbin
/opt	additional user applications (e.g., R, X11, Anaconda)
/sbin	system binaries
/usr	applications and libraries not critical for OS; most user applications
/var	variable data files: logs, print cues, mail spools, etc.

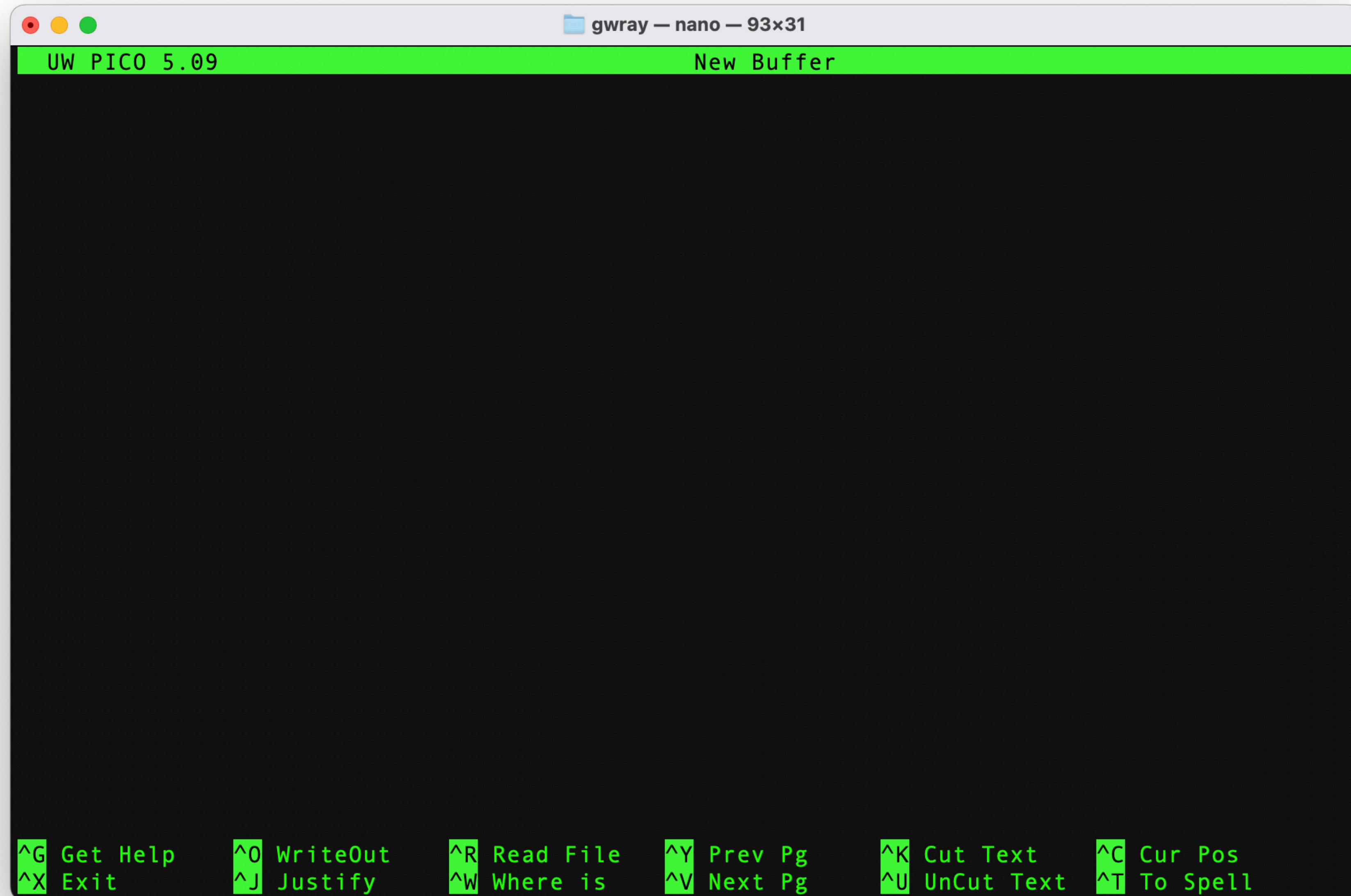
Standard Unix file structure, with more details



Nano text editor

nano interface

Useful for quick edits to small files; opens instantly, provides basic editing functionality



To create/open a file:

```
> nano myfile.txt
```

Essential commands:

```
^O save
```

```
^X exit
```

Additional commands
at bottom of screen

Bash profile

Bash profile

The Bash profile is a configuration file used to customize the user environment

Common uses for the Bash profile:

- List start-up commands: these execute immediately after log-in

- Set aliases

- Customize the prompt

- Set environment variables (more on this in subsequent slides)

How it works:

- A hidden file in the home directory: `~/.bash_profile`

- Contents are executed during log in

- Can be executed later on demand by typing: `source ~/.bash_profile`

Create aliases

Any command can be saved as an **alias**

A useful quick check:

`alias` lists all current aliases

Some examples of useful aliases:

<code>alias ll='ls -a1F'</code>	listing all files, in long format, and show links
<code>alias rms='rm -iv'</code>	"safe mode" for rm; prompts for each deletion
<code>alias grep='grep --color=auto'</code>	highlight matches with color

Customize the bash prompt

The appearance of the prompt is set by a **shell variable**, **PS1**

To change the appearance of the prompt:

```
export PS1='\w \d \t > '
```

 shows pwd, date, time, > character

Some useful prompt components:

- `\W` current directory
- `\w` path to current directory
- `\d` date
- `\t` time
- `\u` user name
- `\h` host name (or `\H` for the full host name)
- `\!` history number of the current command

Unix environment variables

Unix stores several **environment variables**:

Values that control the behavior of the shell and programs running in it

Easily recognizable: all-caps and begin with \$

Some useful environment variables:

<code>\$USER</code>	username
<code>\$HOME</code>	path to your home directory
<code>\$PWD</code>	path to working directory
<code>\$PATH</code>	list of paths the shell uses to find commands and programs

Viewing environment variables:

<code>echo \$PWD</code>	shows the path to the current directory
<code>printenv</code>	shows the current values of all environment variables

THE \$PATH variable

Unix commands only work with files in the current directory unless a path is provided

Yet the commands themselves reside in `/bin`, `/sbin`, and `/lib`

How does the shell know where to find these files?

Unix searches specific locations to find executable files (commands and programs)

Those locations are provided by the `$PATH` variable

Value stored: colon-separated list of paths to search (semicolon in PowerShell)

Working with the `$PATH` variable:

`echo $PATH` shows the list of current search paths

`which grep` uses `$PATH` to find the location of `grep` and sends it to STDOUT

`export PATH="$PATH:/path/to/your/directory"` adds a new path to `$PATH`

Text encodings

Encodings

Computers store 0s and 1s; **encodings** specify how to store human-readable information

ASCII — first widespread encoding and basis for most later ones

Name is an acronym for American Standard Code for Information Interchange

Based on 7 bits, specifies 128 distinct printing and non-printing characters

E.g., `1001000` indicates `<tab>` and `1100100` indicates `A`

Unicode — family of encodings used by internet and nearly every current computer

First 128 characters identical to ASCII (for backward compatibility)

Based on 1-4 bytes, potentially specifying >1,100,000 characters

UTF-8 is most widespread encoding by far; specifies >100,000 characters

Includes mathematical symbols, arrows, scripts for many languages, emoji, etc.

ASCII encoding

b₇ b₆ b₅ b₄ b₃ b₂ b₁ Bits Column Row					0 0 0		0 0 1		0 1 0		0 1 1		1 0 0		1 0 1		1 1 0		1 1 1	
					0	1	2	3	4	5	6	7								
					0	0	0	0	0	@	P	`	p							
					0	0	0	1	1	A	Q	a	q							
					0	0	1	0	2	B	R	b	r							
					0	0	1	1	3	C	S	c	s							
					0	1	0	0	4	D	T	d	t							
					0	1	0	1	5	E	U	e	u							
					0	1	1	0	6	F	V	f	v							
					0	1	1	1	7	G	W	g	w							
					1	0	0	0	8	H	X	h	x							
					1	0	0	1	9	I	Y	i	y							
					1	0	1	0	10	J	Z	j	z							
					1	0	1	1	11	K	[	k	{							
					1	1	0	0	12	L	\	l								
					1	1	0	1	13	M	]	m	}							
					1	1	1	0	14	N	^	n	~							
					1	1	1	1	15	O	_	o	DEL							

sort order:

<tab>

<return>

<space>

symbols/punctuation

numerals

more symbols/punct.

upper case letters

more symbols

lower case letters

more symbols

US-ASCII 1967, the most widely adopted encoding during the early days of computers

UTF-8 encodings

Notation

U+ followed by 4 or 6 **hexadecimal** numerals (left pad if needed)

Values in range **U+0000-U+10FFFF** (vast majority of values are not assigned)

First 2 digits define the **plane**; plane **0** does not require first 2 digits (**00**)

Each plane contains 65,536 potential character encodings (**code points**)

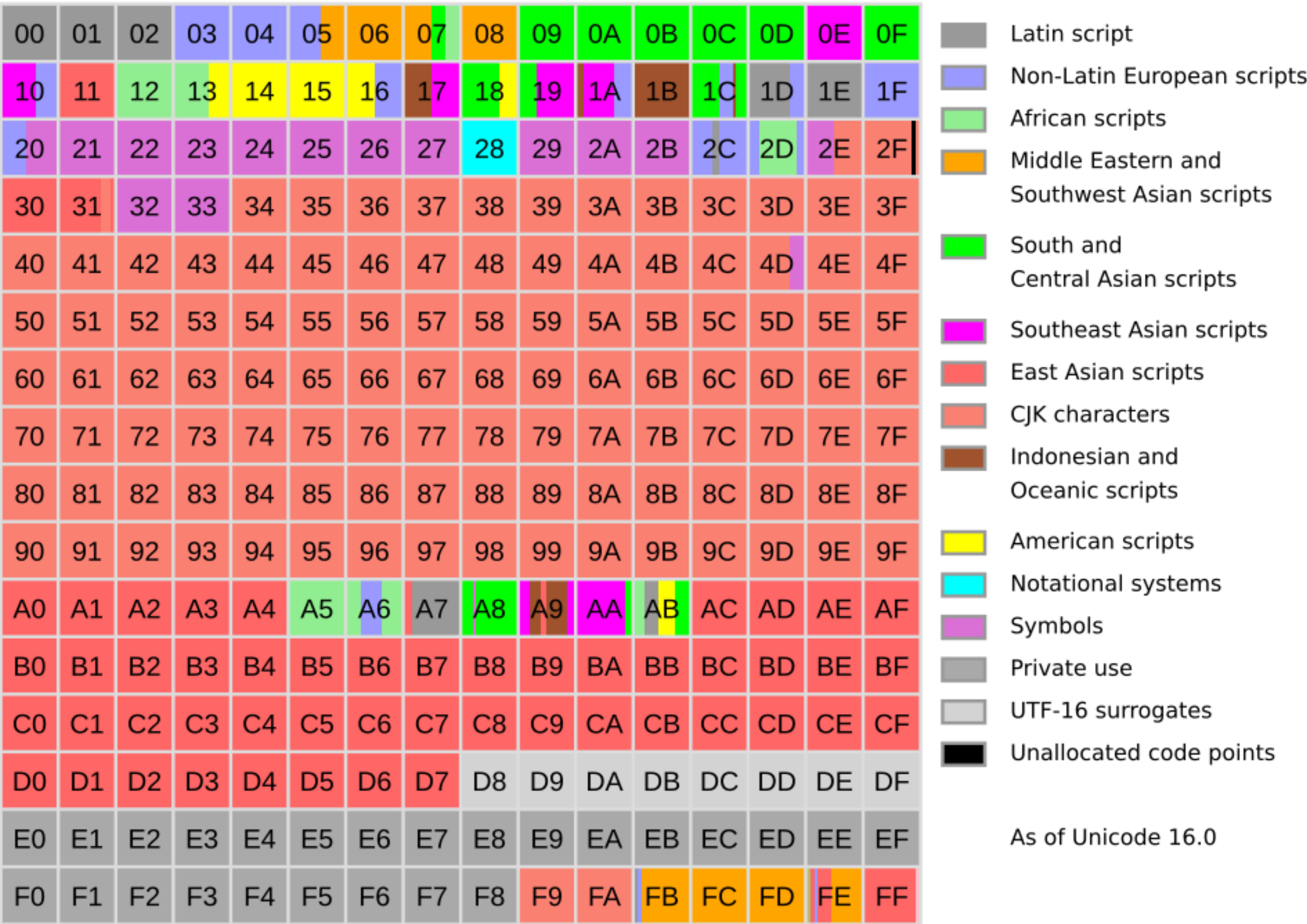
Examples

U+004B	K	Latin lower-case letter K
U+006B	k	Latin lower-case letter K
U+0915	क	Devanagari letter “ka”
U+0643	ك	Arabic letter “kaf”
U+20AC	€	Euro currency symbol
U+01F60E	😎	smiling face with sunglasses (plane 1; requires 6 digits)

By far the most commonly used UTF-8 planes

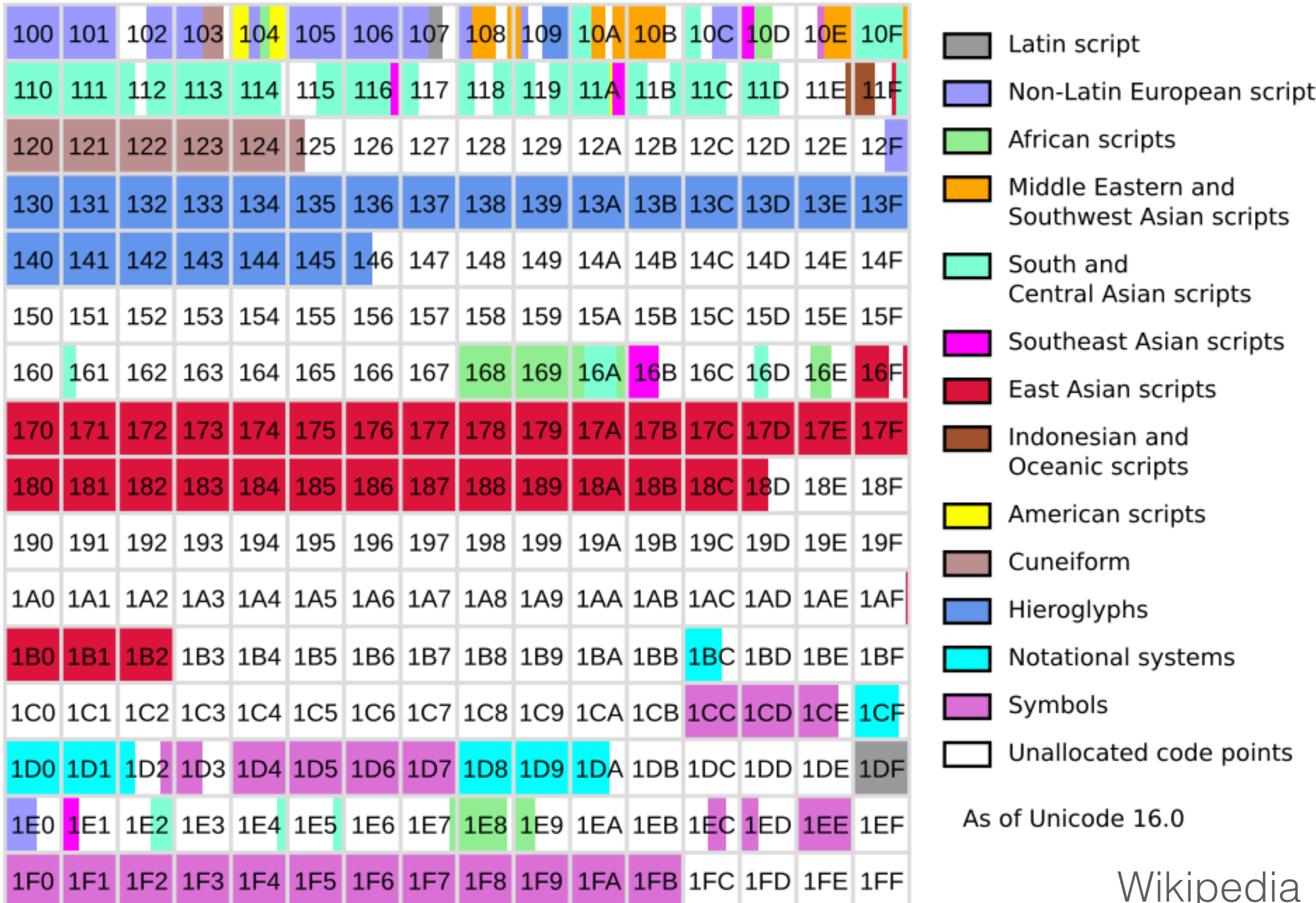
Plane 0

Basic Multilingual



Plane 1

Supplementary Multilingual



Wikipedia

Each square contains up to 256 code points; with many unallocated in planes 1 and above

Unicode links

Official Unicode materials

Unicode v16.0 character code charts: <http://www.unicode.org/charts/>

Unicode standard: <https://www.unicode.org/versions/Unicode16.0.0/>

Useful third-party resources

Shapecatcher: <https://shapecatcher.com/> (search by shape)

Unicode Lookup: <https://unicodelookup.com/> (search by word, character, or code)

EmNudge: <https://unicode.emnudge.dev/> (search by word or code)

r21a's converter: <https://r12a.github.io/app-conversion/> (type in character, get code)

Wikipedia: https://en.wikipedia.org/wiki/List_of_Unicode_characters (browse)

Unicode, expansions, and regular expressions in Unix

Unicode in Unix

Using unicode with **echo**:

```
echo -e '\u03a9'  
echo -e 'Bonjour, He1\00E9ne'  
echo -e 'Devanagari ka: \u0915'  
echo -e 'Here is an emoji: \U01F610'
```

single character; renders as Ω
directly in-line; renders as é
non-Latin character; renders as क
emoji work too; renders as 😊

Using unicode with **grep**:

```
grep '$'\u2122' VoyageOfTheBeagle.txt  
grep '$'\u2020' IOC_14.2.csv
```

note use of \$ before escaping
works with .csv files

Case-insensitive: \U and \u are equivalent; \03a9 and \03A9 are equivalent

Expansions in Unix

Expansions are text or expression substitutions; Unix provides a rich set of expansions

Environment variables

```
echo $SHELL
```

returns single value

```
echo "The path to here is: $PWD"
```

inserts value

Arithmetic expressions: integer math only, but supports a wide range of operators

```
echo $((40 + 2))
```

returns 42

```
echo $((2**8))
```

returns 256 (2 exp 8)

Commands: anything you can type at the command line, including pipes

```
echo "The path to here is: $(pwd)"
```

returns single value

```
echo $(pwd) "contains" $(ls -la | wc -l) files
```

inserts results

Regular expressions in Unix

Regular expressions are integrated into key Unix tools: `grep`, `sed`, and `awk`

The primary exceptions:

- `ls` uses simple globbing: `?` = any 1 character, `*` = any 0 or more characters
- `tr` designed for character-by-character replacement

Unicode with grep

By default, grep accepts only **simple regex** expressions

```
grep "^boat" VoyageOfTheBeagle.txt  
grep -w "wa.e" VoyageOfTheBeagle.txt  
grep -w "wa(k|v)e" VoyageOfTheBeagle.txt
```

anchors work

any character works

alternates do not work

To use **extended regex**, include the `-E` option; includes: `+`, `?`, `|`, `()`, and `{n,m}`

```
grep -E -w"wa(k|v)e" VoyageOfTheBeagle.txt  
grep -E "zo+" VoyageOfTheBeagle.txt  
grep -E "zo{2,3}" VoyageOfTheBeagle.txt
```

alternates and groups work

quantifiers work

quantifiers work

Regular expressions: quick review part 1

Any character: `.`

```
grep 'ca.e' Voyage.txt
```

finds line with `cane`, `case`, etc.

Any character from a set: `[xyz]`

```
grep 'c[au]p' Voyage.txt
```

finds line with `cap`, `cup`, but not `copy`

```
grep 'c[^au]p' Voyage.txt
```

negation: finds `copy`, but not `cap`, `cup`

Alternate strings: `(x|y)`

```
grep -E 'Ta|Va' Voyage.txt
```

finds lines with `Tahiti`, `Vancouver`, etc.

```
grep -E 's(mo|al)' Voyage.txt
```

finds lines with `smooth`, `salmon`, etc.

Regular expressions: quick review part 2

Anchors: ^ and \$

```
grep '^Tahi' VoyageOfTheBeagle.txt
```

lines starting with **Tahiti**, etc.

Quantifiers: ?, *, and +; also {n} and {mn}

```
grep -E 'ATCGTTAA?' yeast_sequence.fasta
```

final **A** repeated 0 or 1 times

```
grep -E 'ATCGTTAA*' yeast_sequence.fasta
```

final **A** repeated 0 or more times

```
grep -E 'ATCGTTAA+' yeast_sequence.fasta
```

final **A** repeated 1 or more times

```
grep -E 'T{7}' yeast_sequence.fasta
```

T repeated exactly 7 times

```
grep -E 'T{7,10}' yeast_sequence.fasta
```

T repeated 7 to 10 times

sed, a stream editor

What is a stream editor?

A **stream editor** automatically carries out a specified task on each line of a file

Non-interactive, in contrast to most text editors

"stream" refers to reading files in groups of lines; very memory-efficient

How **sed** works:

Input stream can come from a file or a pipe

Works line-by-line (similar to grep), hence can work with enormous files

Applies an operation to each line, optionally based on a condition

Multiple operations can be applied in sequence

The input file is not modified

Output is passed to STDOUT, but can be redirected

Uses for sed

Substitution

- Find and replace specific patterns; control over number of substitutions

- Find and replace entire lines based on a pattern or line number

Insertion

- Insert lines based on pattern or line number

- Insert text at the beginning, end, or specific location within lines based on same

Filtering

- Filter lines based on pattern or line number

Deleting

- Delete patterns based on pattern or line number

- Delete lines based on pattern or line number

Using sed

Basic syntax

```
sed options 'command' input_file
```

Enclose the command in single quotes

Multiple input files can be provided; separate file names with spaces

Common options:

- n** suppress automatic line printing (default is to send each line to STDOUT)
- E** use extended regular expressions (similar to grep)
- e** use multiple commands: precede each command with **-e**
- f** read commands from a file (script)
- i** edit in-place (modifies the original file; use with caution!)

Quick reference to common sed commands

<i>command</i>	<i>example</i>	
p print	'2p'	print just line 2
s substitute	's/ant/bat/g'	substitute every "ant" with "bat"
d delete	'\$d'	delete the last line
i insert	'3i Hello'	insert a line with "Hello" before line 3
a append	'3a Hello'	insert a line with "Hello" after line 3
c change	'/ant/c\Hello'	replace lines containing "ant" with "Hello"
q quit	'/ant/q'	print lines up to first instance of "ant"
G print	'G'	insert a blank line after every line

